

Análisis de concurrencia

Uso de hilos y autonomía de las serpientes

El sistema utiliza **concurrencia basada en hilos** para otorgar autonomía a cada serpiente del juego. Cada instancia de SnakeRunner implementa la interfaz Runnable y se ejecuta en un **hilo independiente**, lo que permite que cada serpiente:

- Decida su movimiento de manera autónoma.
- Controle su propia velocidad (normal o turbo).
- Reaccione a eventos del entorno.

El método run() de SnakeRunner mantiene un ciclo de ejecución continuo donde:

1. Se decide un posible giro.
2. Se solicita al tablero el avance mediante board.step(snake).
3. Se ajusta la velocidad del hilo usando Thread.sleep().

Esto permite un comportamiento concurrente real, donde múltiples serpientes pueden interactuar simultáneamente con el mismo tablero compartido.

Posibles condiciones de carrera

Aunque el tablero (Board) protege su estado mediante métodos synchronized, existen **posibles condiciones de carrera** en otros componentes:

1. **Clase Snake**
 - El cuerpo **no está sincronizado**.
 - Si una misma serpiente fuera accedida por más de un hilo, podrían producirse inconsistencias.
 - Solo la variable direction es volatile, lo cual garantiza visibilidad, pero **no exclusión mutua**.
2. **Interacción Snake – Board**
 - Board.step() asume que cada Snake es utilizada por un único hilo.
 - Si dos hilos intentaran mover la misma serpiente simultáneamente, el estado del cuerpo podría corromperse.
3. **GameClock**
 - El tick se ejecuta en un hilo del ScheduledExecutorService.

- Si el tick accede a estructuras compartidas del juego que también usan los SnakeRunner, y estas no están sincronizadas correctamente, puede producirse una condición de carrera.

Colecciones o estructuras no seguras en contexto concurrente

- En Board se utilizan:
 - HashSet (mice, obstacles, turbo)
 - HashMap (teleports)

Estas colecciones **no son thread-safe**, pero su acceso está protegido mediante métodos synchronized, lo que evita accesos concurrentes peligrosos.

- En Snake:
 - ArrayDeque (body) **no es thread-safe**.
 - No existe ningún mecanismo de sincronización sobre esta estructura.

Por lo tanto, la seguridad concurrente de Snake depende estrictamente de que **solo un hilo la modifique**.

Ocurrencias de espera activa (busy-wait) o sincronización innecesaria

- **No existe espera activa (busy-wait)** en el código.
 - Los hilos no utilizan ciclos vacíos para esperar.
 - La temporización se maneja correctamente mediante:
 - Thread.sleep() en SnakeRunner.
 - ScheduledExecutorService en GameClock.
- **Sincronización innecesaria**
 - Los getters de Board están sincronizados y retornan copias defensivas.
 - Esto es seguro, pero puede generar ligera **sobrecarga** si se llaman con alta frecuencia (por ejemplo, desde el renderizado).
 - No obstante, esta decisión prioriza la **consistencia del estado** sobre el rendimiento.